Multiplicación de matrices de forma secuencial y paralela

Presentado por:
Estudiante

2022

Abstract

Por medio del presente documento se muestra el proceso realizado para llevar a cabo el algoritmo de multiplicación de matrices en el lenguaje de programación C, las multiplicaciones fueron realizadas de dos formas: secuencial y paralela, esto con el fin de analizar su desempeño tras ejecutarlo, variando el tamaño de las matrices a multiplicar y la disposición de los recursos del procesador a través de la librería pthread. Se han realizado pruebas ejecutando el algoritmo de forma secuencial, luego se implementaron hilos con la librería pthread y 32 hilos y por último se implementó procesos con forks. Con base en estas pruebas se han obtenido las respectivas conclusiones

Contents

1 Introducción **1**

2 Marco conceptual **2**
2.1 High Performance Computing 2
2.2 Multiplicación matricial 2
2.3 Complejidad Computacional 2
2.4 Programación paralela 2
2.5 Hilos 2
2.6 Speedup 3

3 Marco Contextual **3**
3.1 Características de la máquina 3
3.2 Desarrollo 3

4 Pruebas **4**

5 Conclusiones **4**

6 Bibliografía **5**

1 Introducción

A lo largo de los años la computación ha ido evolucionando hasta el punto de poder resolver problemas de grandes dimensiones en un tiempo más reducido, un ejemplo de estos problemas es la multiplicación de matrices. A pesar de la brevedad para programar este algoritmo en algún lenguaje de programación, "se ha invertido mucho trabajo en hacer que los algoritmos de multiplicación de matrices sean eficientes. [...] Se han diseñado muchos algoritmos diferentes para multiplicar matrices en diferentes tipos de hardware, incluidos sistemas paralelos y distribuidos, donde el trabajo computacional se distribuye en múltiples procesadores" (Qaz Wiki, 2020). En este caso se ha optado por resolver el problema programando dos versiones, una para darle solución de manera secuencial y otra para darle solución de manera paralela. Por otra parte, teniendo en cuenta la definición del speedup (o aceleración) "Herramienta de la Mejora continua que de forma metodológica busca incrementar la velocidad de procesamiento de las máquinas" (Progressa Lean, 2014) se pretende calcular este valor en base a los resultados arrojados tras la ejecución de ambas versiones del algoritmo programadas en el lenguaje C, utilizando los conceptos vistos durante las sesiones del curso HPC.

Finalmente se busca comprobar que, ejecutando el algoritmo con múltiples hilos, se hace posible reducir los tiempos de ejecución del algoritmo, beneficiando a las áreas donde este se aplica, tales como el álgebra lineal o la inteligencia artificial.

2 Marco conceptual

2.1 High Performance Computing

Cuando hablamos de HPC por sus siglas en inglés, o Computación de alto rendimiento en español, "hacemos referencia a un campo de la computación actual que da solución a problemas tecnológicos muy complejos y que involucran un gran volumen de cálculos o de coste computacional." (López, 2017) Para nuestro caso, al aumentar la dimensión de las matrices a multiplicar, esto supone un alto costo computacional, el cual, según el análisis, se ve disminuido gracias al uso de herramientas que permiten, precisamente, disminuir el tiempo de ejecución.

2.2 Multiplicación matricial

Para implementar el algoritmo en el lenguaje C es necesario comprender previamente algunos conceptos, entre ellos, como es el proceso de multiplicar matrices de forma manual, el cual podemos aprender fácilmente gracias a vídeos explicativos y a diversos sitios web, tal como Producto de matrices. Otros sitios en internet nos dan una idea más general de la programación de la multiplicación matricial, como por ejemplo el que se expone a continuación, que ha sido extraído del blog Análisis y diseño de algoritmos. Dadas dos matrices A y B, tales que el número de columnas de la matriz A es igual al número de filas de la matriz B, es decir:

$$A := (a_{ij})_{m\times n} \quad Y \quad B := (b_{ij})_{n\times p}$$

La multiplicación de A por B genera una nueva matriz resultado de la forma: $C = AB := (C_{ij})_{m\times p}$ donde cada elemento C_{ij} está definido por:

$$C_{ij} = \sum_{k=1}^{q} a_{ik} b_{kj}$$

Definición que permite acercarse más a una idea del código ya que de entrada plantea las tres variables a iterar para cada una de las celdas de la matriz resultado.

2.3 Complejidad Computacional

"La Teoría de la Complejidad estudia la eficiencia de los algoritmos en función de los recursos que utiliza en un sistema computacional (usualmente espacio y tiempo)" (Cortéz, 2004). Para este estudio, la complejidad computacional permite observar que en la multiplicación de matrices se gastan bastantes recursos de procesamiento en cuanto a tiempo, es decir, para matrices de dimensiones muy grandes, el procesador estará ocupado bastante tiempo resolviendo la multiplicación, lo que motiva aún más a programar de manera paralela el algoritmo, aprovechando al máximo los recursos del procesador.

2.4 Programación paralela

"La computación paralela es el uso de múltiples recursos computacionales para resolver un problema. Se distingue de la computación secuencial en que varias operaciones pueden ocurrir simultáneamente." (Ferestrepoca, 2019) En nuestro caso de estudio se ejecutará el código de multiplicación de matrices de forma secuencial y de forma paralela, esto con el fin de comparar los resultados obtenidos por cada ejecución y de esta manera poder analizar la eficiencia de la programación paralela.

2.5 Hilos

"La diferencia fundamental entre hilos y procesos es que un proceso dispone de un espacio de memoria separado, mientras que un hilo no. De esta forma los hilos pueden trabajar directamente con las variables creadas por el proceso padre, mientras que los procesos hijos no pueden hacerlo." (Montero & Antunez, 2011) Esta información nos facilita la comprensión del principio aplicado para esta implementación.

2.6 Speedup

El speedup representa la ganancia que se obtiene en la versión paralela de un programa con respecto a su versión secuencial. Para este estudio se ha tomado el speedup como:

$$\text{Speedup} = \frac{T^*(n)}{T_p(N)}$$

Donde $T^*(n)$ hace referencia al tiempo de ejecución secuencial y $T_p(N)$ hace referencia al tiempo de ejecución paralelo.

3 Marco Contextual

3.1 Características de la máquina

Se probó el algoritmo para la multiplicación de matrices generando matrices cuadradas de dimensiones 10, 100, 200, 400, 800, 1600 y 3200 respectivamente. Uno de los algoritmos fue ejecutado de manera secuencial y el otro utilizando hilos. Se analizaron los distintos métodos evaluando los recursos consumidos como el tiempo; se sacaron algunas gráficas para mostrar los resultados obtenidos.

Características del PC donde se realizaron las pruebas:

- Procesador: Intel(R) Core(TM) i7-10700
- Cantidad de núcleos: 8
- Cantidad de subprocesos/hilos: 16
- Frecuencia básica del procesador: 2.90GHz
- Frecuencia Turbo máxima: 4.80GHz
- Ram 16GB DDR4 @ 2400 MHz
- SSD: 240GB - R: 540 MB/s - W: 500 MB/s
- Sistema operativo: Ubuntu 20.04.4 LTS

3.2 Desarrollo

Para iniciar el desarrollo de esta actividad lo primero era generar matrices cuadradas con números random, por lo cual se usó la librería stdlib.h. El tamaño de las matrices cuadradas usadas fueron 10, 100, 200, 400, 800, 1600 y 3200. Con las matrices a multiplicar listas, se pudo aprovechar la implementación en forma secuencial, con hilos y por último con procesos para analizar el comportamiento del algoritmo. El código en C de la multiplicación de matrices utilizando hilos se basa en el código Programación multithread en C (pthread.h) con el cual se comenzó añadiendo las librerías de hilos pthread.h para crear, asignar y juntar las actividades que debe realizar cada hilo y la librería time.h para poder contabilizar el tiempo que demora la ejecución. Para trabajar con time.h de forma correcta, algunas partes del código se basan en el recurso 8 formas de medir el tiempo de ejecución en C / C ++. El código empieza asignando el número de hilos y las variables que almacenarán el tamaño de las filas y las columnas de las matrices que se usarán en la ejecución de la multiplicación. Se pide al usuario que digite el tamaño de la matriz para seguido a esto, con la función malloc reservar memoria para poder trabajar con hilos, posteriormente se procede a la lectura de las matrices y la creación de los hilos a través de funciones implementadas dentro del código. Una vez creados los hilos, se toma el tiempo a través de gettimeofday. Por otra parte, cuando ya está definida la forma en cómo se creará la cantidad de hilos y la unión del trabajo de cada uno, se hace la declaración de la función multiplicación la cual está diseñada para asignar las filas por multiplicar en cada uno de los hilos que la utilicen, además, dentro de esta función se realiza la operación de multiplicación de las matrices por medio de tres ciclos anidados. Finalmente, se escribe el resultado de la multiplicación de matrices hecho por cada hilo, se une a través de la función proporcionada por pthread.h para tener la matriz resultante completa y utilizando nuevamente la función gettimeofday se hace posible calcular el tiempo de ejecución entre la creación de hilos y la unión de sus resultados. La salida de este programa es únicamente el tiempo de ejecución. El código de la multiplicación de matrices secuencial usa las mismas librerías usadas con los hilos, excepto pthread.h, ya que esta vez la ejecución es

secuencial. El código empieza definiendo las variables que almacenarán el tamaño de filas y columnas de la matriz que vamos a usar en la ejecución de la multiplicación, se declaran los arreglos bidimensionales para las matrices a multiplicar siendo estas DoubleA y DoubleB y el arreglo bidimensional Double C para el resultado. Se le pide al usuario que ingrese el tamaño de la matriz y se usa la función malloc para hacer la reserva de memoria para poder ejecutar y se procede a la lectura de las matrices a través de la función read mat, la cual recibe la cantidad de filas y columnas de la matriz. Posteriormente se toma el tiempo con gettimeofday y se hace el llamado a la función matmul cuyos parámetros de entrada son las filas y columnas, que representan la dimensión de las matrices a multiplicar y dentro de esta función se realiza la operación de multiplicación de las matrices por medio de tres ciclos anidados y una vez termina esta función de ejecutarse, se toma el tiempo nuevamente con gettimeofday, con el fin de establecer la diferencia de tiempo entre el inicio y fin de la ejecución. Al igual que en el caso de los hilos, este programa secuencial imprime únicamente el tiempo. Luego de hacer la ejecución de cada multiplicación de matriz con diferentes tamaños y en el caso de los hilos con diferente número de hilos, obtenemos los resultados del tiempo de ejecución de cada uno de estos, lo que permite realizar los análisis y cálculos de la variable speedup para cada uno de los casos. El código fuente resultante de la implementación descrita se encuentra registrado en el enlace: Caso de Estudio Matrices con las respectivas instrucciones para una ejecución correcta.

4 Pruebas

Ahora, para complementar el análisis de los datos y tener una vista un poco más amplia, se han utilizado las tablas para hacer gráficas comparativas que nos permitan visualizar la eficiencia de trabajar con hilos.

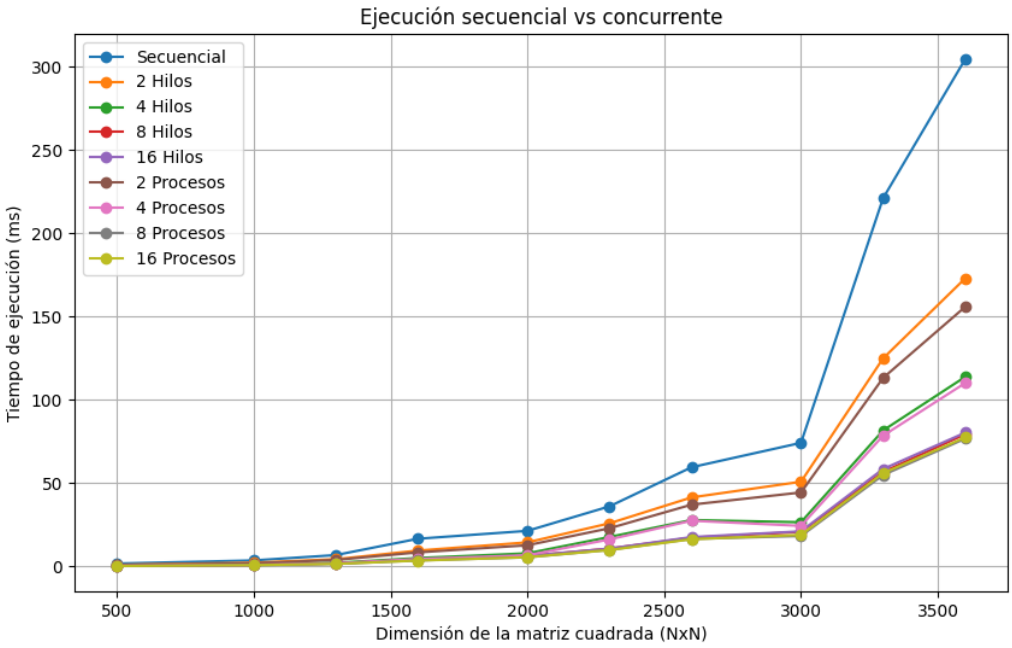

Figure 1: Tiempo de ejecución en función de las dimensiones de la matriz.

A continuación se muestra una gráfica comparativa entre todos los speed up de cada método de paralelización usado, es bueno recordar que el speedup es la división de el tiempo que se tiene en la ejecución secuencial, sobre el tiempo obtenido al paralelizar, por lo que en la gráfica no se incluye en speed up del método secuencial ya que este es una constante igual a 1.

5 Conclusiones

1. Luego de analizar los datos obtenidos, se puede concluir que el uso de hilos es muy necesario para trabajar con matrices de dimensiones superiores a 1200, ya que a partir de este tamaño el tiempo

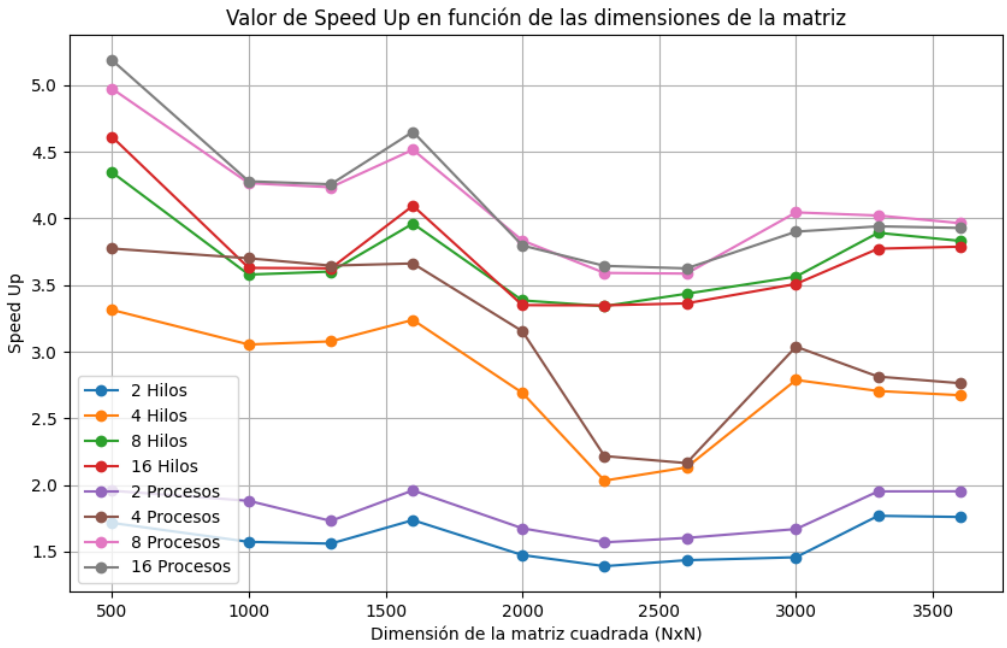

Figure 2: Valor de Speed Up en función de las dimensiones de la matriz.

de ejecución en secuencial es mucho mayor.

2. Los valores de speedup de las ejecuciones con hilos, aumentan proporcionalmente al aumento de los hilos utilizados

3. Usar más de ocho hilos es innecesario, debido a que la diferencia entre los speedup obtenidos con las ejecuciones de 16 y 32 hilos, es muy pequeña.

4. Usar procesos no es tan recomendable, ya que el speed up que se obtiene con pocas o altas dimensiones, no sobresale demasiado frente a paralizaciones superiores a 4 hilos

5. Analizando la gráfica 1, se nota un pico cerca a las ejecuciones con matrices de dimensiones 800, esto, según suposición nuestra, debido al proceso de asignación de memoria que es necesario hacer para poder trabajar con matrices de dimensión superior a 830 aproximadamente

6 Bibliografía

References

[1] Cortéz, A. (2004). TEORÍA DE LA COMPLEJIDAD COMPUTACIONAL Y TEORÍA DE. *Revista De Investigación De Sistemas E Informática*, 1(1), 102-105. Recuperado de: `https://revistasinvestigacion.unmsm.edu.pe/index.php/sistem/article/view/3216`

[2] Ferestrepoca. (2019). Recuperado de: `http://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teoria/index.html`

[3] López, A. d. (2017, 11 30). *Encamina*. From Por una nube sostenible: `https://blogs.encamina.com/por-una-nube-sostenible/introduccion-al-high-performance-computing/`

[4] Montero, L. H., & Antunez, R. R. (2011, 07). *Parallel programming: definitions, mechanisms and trouble*. From `https://www.researchgate.net/publication/274960405_Parallel_programming_definitions_mechanisms_and_trouble`

[5] Progressa Lean. (2014, 11 2). *Progressa Lean*. From https://www.progressalean.com/que-es-speed-up/

[6] Qaz Wiki. (2020). *Qaz Wiki*. From https://es.qaz.wiki/wiki/Matrix_multiplication_algorithm